

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 3 — Solutions

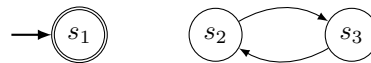
Exercise 3.1 (1+1+1 marks)

Graphically provide an example of a state space with the following property, or argue why such a state space cannot exist:

- (a) A state space that contains cycles but where tree search is still guaranteed to terminate.
- (b) A state space where the optimal expansion order for graph search results in fewer expansions than the optimal expansion order for tree search.
- (c) A state space where tree search and graph search expand the same amount of nodes (when using the same expansion strategy). For this task, your state space must have at least 5 states reachable from the initial state (including the initial state itself) and every solution must have at least length 2.

Solution:

- (a) Possible. Consider the search space below, where the *reachable* state space contains no cycles. Tree search will terminate as it will never explore the unreachable part that contains the cycle.

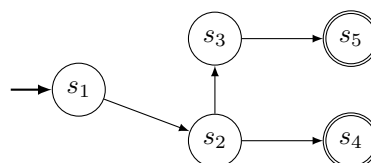


- (b) This depends on whether we require that the state space has a solution.

If the state space should have a solution, it is not possible that the optimal expansion order for graph search results in fewer expansions than the optimal expansion order for tree search: The optimal expansion order for graph search, i.e., the order that expands the minimal amount of nodes before reaching a goal state, is also a valid expansion order for tree search and thus there is an expansion order for tree search that leads to the same amount of expansions.

If the state space has no solution, a graph search would determine this after a finite number of expansions, but in this sense every expansion order that eventually exhausts the reachable state space is equally good. A tree search would require more expansions if some states can be reached on multiple paths, and if there are any reachable cycles, it would not terminate at all. An example for a state space where the optimal expansion order for graph search results in fewer expansions than the optimal expansion order for tree search would be a state space similar to the one for task (a) but where s_1 is not a goal state and where we have an additional transition from s_1 to s_2 .

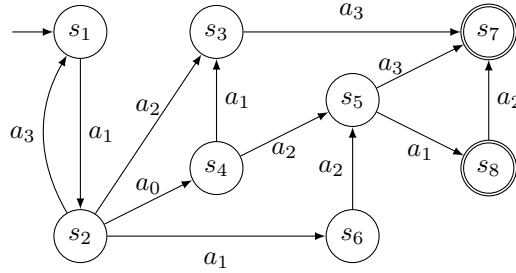
- (c) Since nodes in tree search correspond to *paths* and nodes in graph search correspond to *states*, they expand the same amount of nodes if *each state can be reached by exactly one path*.



Note: Technically we only require those states that are expanded before the goal to be reached by exactly one path. Since we do not know the expansion strategy, the safest way is to simply have all states being reached by exactly one path.

Exercise 3.2 (1.5+1.5 marks)

Consider the following state space with action costs $\text{cost}(a_i) = i$ for all $i \in \{0, 1, 2, 3\}$.



Draw the search trees explored by

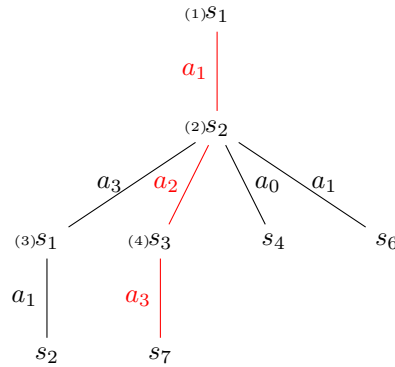
- (a) BFS-Tree (final version) and
- (b) uniform cost search

until a solution is found. Provide the found solutions and indicate the order in which nodes are expanded. If there are multiple states the algorithm could expand next (i.e., there are multiple valid orders of the open list), break ties in favor of lower state indices (e.g., a node with state s_3 is expanded before a node with state s_4).

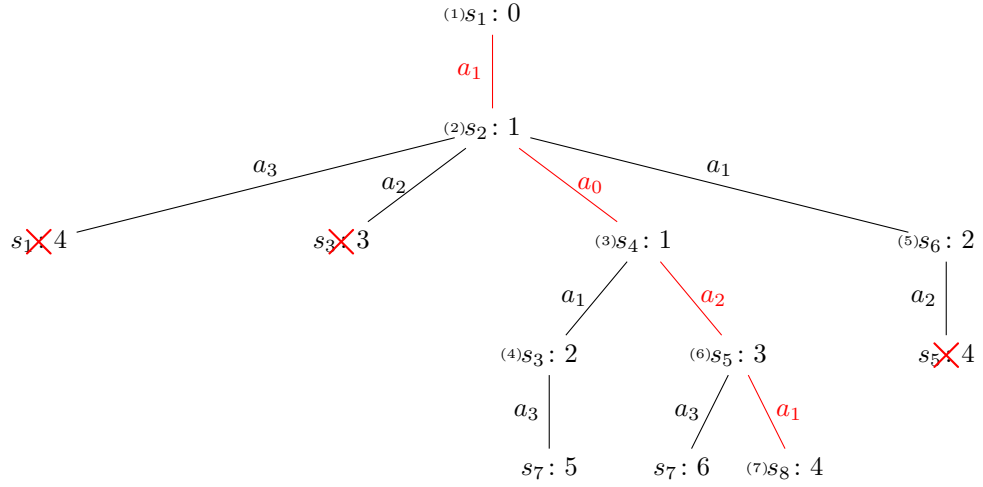
Solution:

Nodes that are expanded are annotated to their left with numbers in parentheses indicating the order of expansion. Paths in the tree corresponding to the found solution are highlighted in red.

- (a) Found solution: $\langle a_1, a_2, a_3 \rangle$

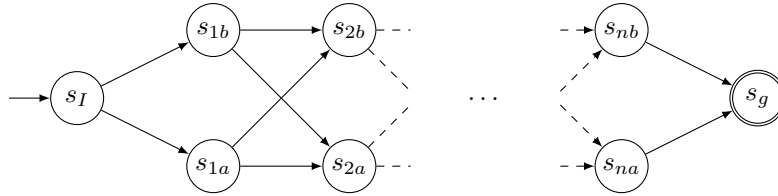


- (b) We annotate g -values after the colon in each node. Nodes representing duplicate states at the time of taking them from the open list are crossed out in red. Found solution: $\langle a_1, a_0, a_2, a_1 \rangle$



Exercise 3.3 (1+1 marks)

Consider a breadth-first search in the following state space:



For the following questions, give an answer as a function of n and justify your answer.

- (a) How many nodes does BFS-Tree (final version) insert into the open list before finding a solution?
- (b) How many nodes does BFS-Graph insert into the open list before finding a solution?

Solution:

- (a) The shallowest solution has length $n + 1$, thus a breadth-first search without duplicate detection must explore (=insert into the open list) all *paths* with length up to n . It then finds that s_g is a goal state when expanding the first state of layer n . There is one path with length 0 (inserting the node representing the initial state), and 2^i paths with length i (for $1 \leq i \leq n$). This yields a total of $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ nodes inserted into the open list.
- (b) A breadth-first search with duplicate detection must explore all *states* which are n steps or less away from the initial state. There is one state (the initial state) that is 0 steps away, and 2 states that are i steps away (for $1 \leq i \leq n$). This yields a total of $1 + (\sum_{i=1}^n 2) = 2n + 1$ nodes inserted into the open list.

Exercise 3.4 (1+1 marks)

Answer the following questions in 1–2 sentences each.

- (a) Why is early goal testing a good idea for breadth-first search but not for uniform cost search?
- (b) Why is a tree variant of uniform cost search not even semi-complete?

Solution:

- (a) Testing for goal states early guarantees to find the same solution as testing for goal states later with BFS, so we can avoid the generation of potentially many additional successor states by testing for goal states early. The same is not true for uniform cost search where solutions may differ depending on whether early goal testing is used or not, and more importantly we lose the optimality guarantee when testing for goal states early.
- (b) Uniform cost search progresses in order of minimal g -value. In contrast to BFS, uniform cost search does not guarantee to eventually expand every generated state, as it is possible to get stuck in a cycle of cost 0 which does not increase g -values.